

claude-windows / 9b717b4b-673e-43c5-b3f4-33099e782c45

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9b717b4b-673e-43c5-b3f4-33099e782c45\subagents\workflows\wf_df7e49b6-842\agent-a4782e5c758144c2e.jsonl`  
- SHA-256: `48ac637c70369dc5cf25a7e3b6df65632ef9fb245f3b55d16ac75ea00febbce9`  
- Source modified: `2026-06-08T08:01:27+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_df7e49b6-842`  
- session_id: `9b717b4b-673e-43c5-b3f4-33099e782c45`
```

Transcript

1. user (2026-06-08T07:56:34.649Z)

You are researching VLC 3.x Lua *extension* internals. Use web search and web fetch (load WebSearch/WebFetch via ToolSearch if needed).

QUESTION: In VLC's "Tools -> Plugins and extensions" window, when a user selects a Lua extension and opens its "About" dialog (window titled "About <name>.lua", showing "Type: Extensions" and the body text "Lua script"), what text is shown and where does it come from?

Specifically determine:

1. Does the descriptor() return table's `description` / `shortdesc` field populate THAT About dialog, or is that dialog showing a generic per-module Lua help string ("Lua script") that an individual extension script cannot control?
2. Where IS the descriptor `description`/`shortdesc` actually surfaced in the VLC UI (e.g. the "Active Extensions" tab list)?
3. What is the MOST RELIABLE way for a VLC 3.x Lua extension to show author-controlled "how to use" help text to the user? The extension already builds a persistent `vlc.dialog`. Confirm whether an in-dialog Help button and/or an `add\_html` widget can render multi-line help, and list the relevant vlc.dialog API methods (add_html, add_label, add_button, add_text_input, etc.).

Consult: the VideoLAN wiki (wiki.videolan.org documentation on Lua "Extensions" and the dialog/widget API), the VLC source (modules/lua/extension.c, modules/lua/libs/dialog.c, modules/lua/libs/objects.c), and VLC forum threads. Provide URLs. Be precise; explicitly flag uncertainty; do NOT fabricate API methods.

2. assistant (2026-06-08T07:56:36.942Z)

I'll research VLC 3.x Lua extension internals. Let me start by loading the web tools and examining the VLC source code.

3. assistant (2026-06-08T07:56:39.805Z)

I have the source files referenced in the question. Let me fetch the VLC source code directly from the official git repository, plus the wiki documentation. Let me start in parallel.

4. user (2026-06-08T07:56:49.770Z)

Lua Extension Descriptor Storage

(1) Descriptor Table Fields and Storage

The `descriptor()` function's return table fields are extracted and stored on the `extension\_t` struct:

```
- title: Stored in p_ext->psz_title. Code: "p_ext->psz_title = strdup(
luaL_checkstring( L, -1 ) )"
- description: Stored in p_ext->psz_description. Code: "p_ext->psz_description =
luaL_strdupornull( L, -1 )"
- shortdesc: Stored in p_ext->psz_shortdescription. Code: "p_ext->psz_shortdescription
= luaL_strdupornull( L, -1 )"
- author, url, version, icon: Also extracted and stored on the extension
struct
```

There is no explicit `name` field extracted from the descriptor table; the script filename becomes `p_ext->psz_name`.

(2) Module-Level Help Text

The code contains **no** `vlc_module` definition or generic module help string registration. The extension system dynamically loads Lua scripts without a static module declaration.

(3) About Dialog Text

The provided code does not show explicit "About" dialog generation or hardcoded strings like `"Lua script"` or `"Type: Extensions"`. These UI elements would be implemented in separate VLC interface code not included in this file.

5. user (2026-06-08T07:56:53.678Z)

VLC Dialog Widget Methods

Widget Creation Methods

All widget creation methods are registered in `vlclua_dialog_reg`:

```
```c
{ "add_button", vlclua_dialog_add_button },
{ "add_label", vlclua_dialog_add_label },
{ "add_html", vlclua_dialog_add_html },
{ "add_text_input", vlclua_dialog_add_text_input },
{ "add_password", vlclua_dialog_add_password },
{ "add_check_box", vlclua_dialog_add_check_box },
{ "add_dropdown", vlclua_dialog_add_dropdown },
{ "add_list", vlclua_dialog_add_list },
{ "add_image", vlclua_dialog_add_image },
{ "add_spin_icon", vlclua_dialog_add_spin_icon },
```
```

Complete Dialog-Level Methods:

```
```c
{ "show", vlclua_dialog_show },
{ "hide", vlclua_dialog_hide },
{ "delete", vlclua_dialog_delete },
{ "set_title", vlclua_dialog_set_title },
{ "update", vlclua_dialog_update },
{ "del_widget", vlclua_dialog_delete_widget },
```
```

HTML Widget Confirmation

Yes, `add_html` exists. The inline function confirms:

```
```c
static inline int vlclua_dialog_add_html(lua_State *L)
{
 return vlclua_dialog_add_text_inner(L, EXTENSION_WIDGET_HTML);
}
```

```
}
...
```

However, the code provides **no explicit documentation** about multi-line or rich HTML rendering capabilities. The implementation delegates to `vlclua_dialog_add_text_inner()` with type `EXTENSION_WIDGET_HTML`, but rendering details remain undefined in this source file.

## Widget Methods

Widget-level methods (from `vlclua_widget_reg`):

```
```c  
{ "set_text", vlclua_widget_set_text },  
{ "get_text", vlclua_widget_get_text },  
{ "set_checked", vlclua_widget_set_checked },  
{ "get_checked", vlclua_widget_get_checked },  
{ "add_value", vlclua_widget_add_value },  
{ "get_value", vlclua_widget_get_value },  
{ "clear", vlclua_widget_clear },  
{ "get_selection", vlclua_widget_get_selection },  
{ "animate", vlclua_widget_animate },  
{ "stop", vlclua_widget_stop },  
```
```

### 6. assistant (2026-06-08T07:56:58.322Z)

Now I have the core source facts. The About dialog with "Lua script" text isn't in `extension.c`, so it must come from the module descriptor. Let me find where "Lua script" and "Type: Extensions" come from. This About dialog is likely the Qt module info dialog reading from `module_t`. Let me check the Lua module's main descriptor and the Qt plugins dialog.

### 7. user (2026-06-08T07:57:04.854Z)

I cannot find the `vlc_module_begin()` ... `vlc_module_end()` declaration in the provided file content. The file shown is `extension.c`, which contains implementation code but does not include the module metadata section you're asking about. That metadata would typically be found in a separate module definition or at the beginning of a different source file in the VLC project structure.

To locate the specific lines with `set_capability`, `set_description`, `add_shortcut`, and `set_help`, you would need to examine the module's initialization file or header file that declares these settings.

### 8. user (2026-06-08T07:57:05.508Z)

Web search results for query: "VLC Lua extension "About" dialog "Lua script" module help plugins extensions window"

Links: [{"title": "Dialog Module - VLC Lua Docs", "url": "https://vlc.verg.ca/m/dialog/"}, {"title": "Extension Scripts - VLC Lua Docs", "url": "https://vlc.verg.ca/t/extensions/"}, {"title": "vlc-lua-docs/t/extensions/index.md at master · verghost/vlc-lua-docs", "url": "https://github.com/verghost/vlc-lua-docs/blob/master/t/extensions/index.md"}, {"title": "How to use a dialog in an interface? - The VideoLAN Forums", "url": "https://forum.videolan.org/viewtopic.php?t=113706"}, {"title": "vlc/share/lua/extensions at master · videolan/vlc", "url": "https://github.com/videolan/vlc/tree/master/share/lua/extensions"}, {"title": "Lua extensions: selected item API for dialog dropdown widget (#6548) · Issues · VideoLAN / VLC · GitLab", "url": "https://trac.videolan.org/vlc/ticket/6548"}, {"title": "vlc/share/lua at master · videolan/vlc", "url": "https://github.com/videolan/vlc/tree/master/share/lua"}, {"title": "[Solved] Creating a VLC Plugin/Extension with Lua, for Communicating between VLC and AutoIt - AutoIt General Help and Support - AutoIt Forums", "url": "https://www.autoitscript.com/forum/topic/170144-solved-creating-a-vlc-pluginextension-with-lua-for-communicating-between-vlc-and-autoit/"}, {"title": "VLC Lua Docs", "url": "https://vlc.verg.ca/"}]

Based on the search results, I found information about VLC Lua extensions and dialogs:

Extension scripts are the only kind that have access to the dialog lua module, which allows developers to create a new dialog window. You can create a new UI dialog using ``vlc.dialog("My VLC Extension")`` with a human-readable title, and then show this dialog with ``d:show()``.

When extension scripts are started, VLC looks for a few functions, some of which are necessary for the script to continue execution, and these functions might be required to accept parameters and/or return specific values. In extensions, the descriptor function is required for the extension to be registered by VLC.

Widgets are small components that provide utility and UI of the dialog window and are added by calling the `add_` methods of the dialog object.

The search results point to the VLC Lua documentation which covers the Dialog Module and Extension Scripts that you're interested in learning more about.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

#### 9. assistant (2026-06-08T07:57:13.516Z)

The `extension.c` on the lua module is the "extension" capability. The "About" dialog showing "Lua script" gets the module description. Let me look at the main lua module file (`modules/lua/vlc.c`) which has the ``vlc_module_begin`` with descriptions, and check the Qt plugins dialog code to understand what populates "About <name>.lua".

#### 10. user (2026-06-08T07:57:19.718Z)

## Lua Extension Submodule

The extension submodule contains:

```
```c
add_submodule ( )
    set_shortcode( N_("Lua Extension") )
    set_description( N_("Lua Extension") )
    add_shortcode( "luaextension" )
    set_capability( "extension", 1 )
    set_callbacks( Open_Extension, Close_Extension )
```
```

The ``set_description`` string for the extension submodule is: `***Lua Extension***`

## String Literals Containing "Lua"

- "Lua interpreter"
- "Lua interface"
- "Lua HTTP"
- "Lua CLI"
- "Lua Telnet"
- "Lua Meta Fetcher"
- "Lua Meta Reader"
- "Lua Playlist"
- "Lua Art"
- "Lua Extension"
- "Lua SD Module"

#### 11. user (2026-06-08T07:57:22.280Z)

## descriptor() Function Fields and UI Placement

According to the documentation, the `descriptor()` function returns a table with metadata about an extension. The document specifically mentions where `shortdesc` appears:

```
> "If this is not provided, then the first line of description will be used for short
description in _Tools_?_Plugins and extensions_ menu"
```

This indicates that the short description is displayed in the Tools menu under Plugins and extensions. However, the documentation does not specify where other fields like `title`, `version`, `author`, `url`, `description`, `icon`, or `capabilities` appear in the VLC interface. It also does not mention an "About" dialog or an "Active Extensions" tab for these fields.

## 12. assistant (2026-06-08T07:57:27.412Z)

Now I have a clearer picture. The "About" dialog showing "Lua script" is interesting because the module description is "Lua Extension", not "Lua script". The "Lua script" text and "About <name>.lua" window is the Qt plugins dialog. Let me examine the Qt `plugin_dialog` / `MetaTab` code and confirm what populates the About dialog. The window title "About <name>.lua" and "Type: Extensions" suggests this is the Qt plugins tab showing per-plugin info from `module_t`, where the extension is registered as a plugin named after the script file.

Let me look at the Qt plugins dialog source and the addons/plugin tab.

## 13. user (2026-06-08T07:57:33.919Z)

Web search results for query: "VLC Qt plugins dialog "Type:" module description help "About" mediainfo PluginTab source"

Links: [{"title": "vlc/modules/gui/qt/qt.cpp at master · videolan/vlc", "url": "https://github.com/videolan/vlc/blob/master/modules/gui/qt/qt.cpp"}, {"title": "Debian -- Details of package vlc-plugin-qt in sid", "url": "https://packages.debian.org/sid/vlc-plugin-qt"}, {"title": ".dotfiles/.config/vlc/vlc-qt-interface.conf at master · rafaqz/.dotfiles", "url": "https://github.com/rafaqz/.dotfiles/blob/master/.config/vlc/vlc-qt-interface.conf"}, {"title": "Quickly create a fully functional cross-platform media player - VLC-Qt", "url": "https://vlc-qt.tano.si/"}, {"title": "GitHub - vlc-qt/vlc-qt: VLC-Qt - a simple library to connect Qt application with libVLC", "url": "https://github.com/vlc-qt/vlc-qt"}, {"title": "show VLC (Direct3D output) dialog box, how to disable it? · Issue #118 · vlc-qt/vlc-qt", "url": "https://github.com/vlc-qt/vlc-qt/issues/118"}, {"title": "Qt Interface - VideoLAN Wiki", "url": "https://wiki.videolan.org/Qt\_Interface"}, {"title": "GitHub - anubhavrohatgi/VLC-QT-plugin: Example VLC plugin interfaced through QT project", "url": "https://github.com/anubhavrohatgi/VLC-QT-plugin"}]

I've completed a web search for your query. The search returned results related to VLC's Qt interface and plugins, including:

- Information about VLC's Qt plugin architecture and its implementation in the VLC source code
- References to the Qt interface in VLC (from the VideoLAN wiki)
- VLC-Qt library documentation (a separate library for connecting Qt with libVLC)
- Configuration files for VLC that reference the mediainfo component
- Various GitHub repositories related to VLC's Qt integration

However, the search results don't contain specific information about the exact dialog interface elements you're asking about (the "Type:" module description, "About" tab, PluginTab source code, etc.). For more detailed technical information about VLC's Qt plugins dialog implementation, you may want to:

1. Visit the official VLC source code repository directly at <https://code.videolan.org/videolan/vlc>
2. Look in the `modules/gui/qt` directory for dialog-related source files
3. Search for "PluginTab" or "plugins dialog" in the VLC codebase

Would you like me to perform a more specific search, or do you need information about a particular aspect of VLC's Qt plugin system?

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

14. user (2026-06-08T07:57:36.973Z)

## Dialog Widget Implementation Analysis

### vlclua\_dialog\_add\_text\_inner Implementation

```
```c
static int vlclua_dialog_add_text_inner( lua_State *L, int i_type )
{
    if( !lua_isstring( L, 2 ) && !lua_isnil( L, 2 ) )
        return luaL_error( L, "dialog:add_text_input usage: (text = nil)" );

    extension_widget_t *p_widget = calloc( 1, sizeof( extension_widget_t ) );
    p_widget->type = i_type;
    if( !lua_isnil( L, 2 ) )
        p_widget->psz_text = strdup( luaL_checkstring( L, 2 ) );

    return vlclua_create_widget_inner( L, 1, p_widget );
}
```
```

### Widget Type Mappings

```
- **add_label**: `EXTENSION_WIDGET_LABEL`
- **add_html**: `EXTENSION_WIDGET_HTML`
- **add_text_input**: `EXTENSION_WIDGET_TEXT_FIELD`
- **add_password**: `EXTENSION_WIDGET_PASSWORD`
- **add_button**: `EXTENSION_WIDGET_BUTTON`
```

Both add\_html and add\_label accept optional string text parameters (text may be nil).

### Registration Tables

```
vlclua_dialog_reg[]:
```c
{ "show", vlclua_dialog_show },
{ "hide", vlclua_dialog_hide },
{ "delete", vlclua_dialog_delete },
{ "set_title", vlclua_dialog_set_title },
{ "update", vlclua_dialog_update },
{ "add_button", vlclua_dialog_add_button },
{ "add_label", vlclua_dialog_add_label },
{ "add_html", vlclua_dialog_add_html },
{ "add_text_input", vlclua_dialog_add_text_input },
{ "add_password", vlclua_dialog_add_password },
{ "add_check_box", vlclua_dialog_add_check_box },
{ "add_dropdown", vlclua_dialog_add_dropdown },
{ "add_list", vlclua_dialog_add_list },
{ "add_image", vlclua_dialog_add_image },
{ "add_spin_icon", vlclua_dialog_add_spin_icon },
{ "del_widget", vlclua_dialog_delete_widget },
{ NULL, NULL }
```

vlclua_widget_reg[]:
```c
{ "set_text", vlclua_widget_set_text },
{ "get_text", vlclua_widget_get_text },
{ "set_checked", vlclua_widget_set_checked },
{ "get_checked", vlclua_widget_get_checked },
```

```
{ "add_value", vlclua_widget_add_value },
{ "get_value", vlclua_widget_get_value },
{ "clear", vlclua_widget_clear },
{ "get_selection", vlclua_widget_get_selection },
{ "animate", vlclua_widget_animate },
{ "stop", vlclua_widget_stop },
{ NULL, NULL }
}
```

15. assistant (2026-06-08T07:57:44.867Z)

I now have the full dialog API confirmed. Let me confirm the Qt "About" dialog source ? the window titled "About <name>.lua" with "Type: Extensions" and body "Lua script". This is the Qt `plugin\_dialog` / addons module info. Let me find the Qt source that builds this dialog and what the "Lua script" body text maps to. The key candidate: the simple plugins dialog (`SimplePrefsCat` no) ? actually it's the `PluginDialog` / `PluginTab` in `modules/gui/qt/dialogs/plugins.cpp`. The "Type" field maps to module capability, and the description body comes from `module_get_help()` or `module_get_name()`.

16. user (2026-06-08T07:57:52.400Z)

Analysis of Plugin/Extension "About" Dialogs

(1) Classes Building "About" Dialogs

Two classes construct these dialogs:

- `ExtensionInfoDialog`: For the Extensions tab
- `AddonInfoDialog`: For the Add-ons tab

Neither appears in the Plugins tab code shown.

(2) ExtensionInfoDialog Structure

Dialog title construction:

```
...
setWindowTitle( qtr( "About" ) + " " + index.data(Qt::DisplayRole).toString() );
...
```

Type field and description:

The code shown does NOT include a "Type:" field for extensions. Instead, it displays:

- Version: `index.data(ExtensionListModel::VersionRole)`
- Author: `index.data(ExtensionListModel::AuthorRole)`
- Summary text: `index.data(ExtensionListModel::SummaryRole)`

No VLC module functions are called. The dialog retrieves data purely from model indices, not from `module_get_help()`, `module_get_name()`, or capability functions.

(3) AddonInfoDialog Fields

Dialog title:

```
...
setWindowTitle( qtr( "About" ) + " " + index.data(Qt::DisplayRole).toString() );
...
```

Description display:

```
...
textContent->append( QString("%1<br/>\n")
    .arg( index.data(AddonsListModel::SummaryRole).toString() ) );
...
```

Type field explicitly shown:

```
...
```

```
textContent->append( QString("<b>%1:</b> %2<br/>")
    .arg( qtr("Type") ).arg( type ) );
...
```

Extension/addon descriptions derive from `SummaryRole` and `DescriptionRole` model data, not from module introspection functions.

17. assistant (2026-06-08T07:58:01.833Z)

Interesting ? there are two distinct dialogs. The user described a dialog with "Type: Extensions" and body "Lua script". The `ExtensionInfoDialog` does NOT show "Type:" and shows Summary/Version/Author. But the `AddonInfoDialog` DOES show "Type:" and a description.

Wait ? but the user says the window title is "About <name>.lua" and "Type: Extensions". The word "Extensions" as a *type value* and the body "Lua script" strongly suggests this is actually the **Qt simple plugins/modules info** dialog that lists ALL modules (via `module\_list\_get`), where the extension is registered as a module. Let me look at the `PluginTab` class which lists modules and the per-module about. The "Type" value "Extensions" = the capability, and "Lua script" = the module shortname/description.

Let me re-fetch with focus on the PluginTab and module-based dialog, plus the ExtensionListModel roles to be precise about SummaryRole.

18. user (2026-06-08T07:58:07.747Z)

PluginTab Analysis

(1) Module Enumeration and Columns

The PluginTab enumerates modules using `module\_list\_get()`:

```
```cpp
module_t **p_list = module_list_get(&count);
...
```

It displays three columns:

```
```cpp
QStringList headerNames;
headerNames << qtr("Name") << qtr("Capability" ) << qtr( "Score" );
...
```

The data comes from:

- `module\_get\_name( p\_module, true )` for Name
- `module\_get\_capability( p\_module )` for Capability
- `module\_get\_score( p\_module )` for Score

(2) Double-Click / About Dialog

****No.**** There is no double-click handler or "About" action for plugin rows in the PluginTab class. The code shows no dialog opening mechanism for individual plugins. This differentiates it from the ExtensionTab and AddonsTab.

(3) PluginTab Data Sources

The PluginTab uses only these three module functions:

```
```cpp
qfu(module_get_name(p_module, true))
qfu(module_get_capability(p_module))
QString::number(module_get_score(p_module))
...
```

No `module\_get\_help()` or description retrieval appears in PluginTab.

## (4) ExtensionListModel Roles

The ExtensionCopy class defines roles mapping to extension fields:

```
```cpp
case SummaryRole:
    return shortdesc;
...
case DescriptionRole:
    return qfu( p_ext->psz_description );
...
```
```

In ExtensionInfoDialog, the summary text comes from **shortdescription**:

```
```cpp
label->setText( index.data(ExtensionListModel::SummaryRole).toString() );
...
```
```

19. user (2026-06-08T07:58:09.079Z)

## Enum Role Definitions and Class Declarations

### ExtensionListModel Roles

The enumeration within `ExtensionListModel` defines these roles:

```
- `SummaryRole = Qt::UserRole`
- `VersionRole`
- `AuthorRole`
- `LinkRole`
- `FilenameRole`
```

### AddonsListModel Roles

This derived model extends the roles with:

```
- `TypeRole = FilenameRole + 1`
- `DescriptionRole`
- `UUIDRole`
- `FlagsRole`
- `StateRole`
- `DownloadsCountRole`
- `ScoreRole`
```

### Class Declarations

The header declares these tab and dialog classes:

```
- `class PluginTab : public QVLCFrame`
- `class ExtensionTab : public QVLCFrame`
- `class AddonsTab : public QVLCFrame`
- `class ExtensionInfoDialog : public QVLCDialog`
- `class AddonInfoDialog : public QVLCDialog`
- `class ExtensionListModel : public QAbstractListModel`
```

### Description Handling

The provided header does not contain a role explicitly labeled as representing short descriptions. The `SummaryRole` and `DescriptionRole` appear to serve descriptive purposes, with the latter belonging specifically to the addons model rather than the base extension model.

20. assistant (2026-06-08T07:58:18.103Z)

This is very clarifying. The Qt PluginTab (the list of all compiled modules) has NO per-plugin "About" dialog. The ExtensionInfoDialog shows Summary (=shortdescription), Version, Author ? and does NOT show "Type:".

But the user explicitly describes a dialog titled "About <name>.lua" with "Type: Extensions" and body "Lua script". This combination ? "Type: Extensions" + body "Lua script" + title ending in ".lua" ? does not match the Qt ExtensionInfoDialog. It strongly matches a **different VLC GUI**: this is the behavior of older VLC or a different interface. Actually, "Type: Extensions" with

value being a category and "Lua script" as a help string points to the **module about** where name = script filename (e.g. `time.lua`), capability registered, and `module\_get\_help()` returns "Lua script".

Let me reconsider. The extension is loaded by `Open\_Extension`. Each extension script is registered. Let me check whether extensions get registered as individual modules with help "Lua script". Actually ? the critical insight: when an extension's `descriptor()` doesn't yield certain fields, what's the fallback? And there's a possibility this is the **macOS** VLC interface or an older Qt. Let me verify by searching the actual string "Lua script" in VLC source and checking the macosx VLCMain / VLCPluginDialog.

## 21. user (2026-06-08T07:58:26.318Z)

Web search results for query: "VLC source "Lua script" extension module help set\_help "Lua script" share/lua"

Links: [{"title": "VLC Lua Docs", "url": "https://vlc.verg.ca/"}, {"title": "vlc/share/lua at master · videolan/vlc", "url": "https://github.com/videolan/vlc/tree/master/share/lua"}, {"title": "[Solved] Creating a VLC Plugin/Extension with Lua, for Communicating between VLC and AutoIt - AutoIt General Help and Support - AutoIt Forums", "url": "https://www.autoitscript.com/forum/topic/170144-solved-creating-a-vlc-pluginextension-with-lua-for-communicating-between-vlc-and-autoit/"}, {"title": "Documentation:Building Lua Playlist Scripts - VideoLAN Wiki", "url": "https://wiki.videolan.org/Documentation:Building\_Lua\_Playlist\_Scripts/"}, {"title": "vlc-lua-docs/index.md at master · verghost/vlc-lua-docs", "url": "https://github.com/verghost/vlc-lua-docs/blob/master/index.md"}, {"title": "Extension Scripts - VLC Lua Docs", "url": "https://vlc.verg.ca/t/extensions/"}, {"title": "GitHub - nima64/VLC-3-lua-extension-guide: This a documentation for writing extensions in lua for vlc 3, as the current docs online is deprecated. · GitHub", "url": "https://github.com/nima64/VLC-3-lua-extension-guide"}, {"title": "How to Install a VLC Media Player Addon .lua", "url": "https://www.vlchelp.com/install-vlc-media-player-addon/"}, {"title": "Documentation:Modules/lua - VideoLAN Wiki", "url": "https://wiki.videolan.org/Documentation:Modules/lua/"}]

Based on the web search results, I found relevant information about VLC Lua scripts and extensions. Here are the key findings:

## VLC Lua Script Structure

VLC provides modules that are found in VLC\_INSTALL\_PATH/lua/modules/ and must be explicitly imported into lua code using the built-in require function.

## Extension Scripts and Help Functions

Extension scripts are smaller applications that "extend" VLC functionality, and extension scripts are the only kind that have access to the dialog lua module, which allows developers to create a new dialog window.

The descriptor function is one of the most important special functions expected in most VLC Lua scripts, and in extensions, it is required for the extension to be registered by VLC, and this function only needs to return a table of information, which can have the following fields including title.

## Script Installation Locations

Lua scripts are tried in alphabetical order in the user's VLC config directory lua/{playlist,meta,intf}/ subdirectory on Windows and Mac OS X or in the user's local share directory (~/.local/share/vlc/lua/... on linux), then in the global VLC lua/{playlist,meta,intf}/ directory.

The search results provide comprehensive documentation about VLC's Lua scripting system, including information about modules, extension scripts, and the descriptor function that helps

set up extension metadata, though the specific `set\_help` function you mentioned wasn't directly found in these results.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

## 22. user (2026-06-08T07:58:32.154Z)

The server returned HTTP 504 Gateway Timeout.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

## 23. assistant (2026-06-08T07:58:36.966Z)

Let me use `gh` to search the VLC source for the exact "Lua script" string, and check the macOS GUI which is the most likely place where "Type: Extensions" + "Lua script" appears (the user's described dialog format matches macOS VLC's "Information" / plugin About style, or possibly an older Qt). Let me also reconsider: this could be the macOS `VLCPlaylistInfo` / about-this-extension.

## 24. user (2026-06-08T07:58:39.785Z)

```
share/lua/intf/luac.lua
NEWS
share/lua/README.txt
modules/lua/extension.c
modules/lua/services_discovery.c
modules/misc/addons/fsstorage.c
doc/ldoc-config.ld
modules/lua/vlc.c
configure.ac
extras/package/macosx/package.mak
modules/lua/stream_filter.c
modules/lua/libs/variables.c
share/lua/sd/README.txt
modules/lua/meta.c
```

## 25. assistant (2026-06-08T07:58:45.585Z)

`extension.c` does contain "Lua script". My earlier WebFetch missed it because the model summarized rather than grepping. Let me get the exact lines from extension.c and vlc.c via gh raw content, and check the macOS plugin/about dialog. The "Type: Extensions" + ".lua" + "Lua script" pattern is most likely the \*\*macOS\*\* VLC interface. Let me get precise content.

## 26. user (2026-06-08T07:58:46.904Z)

```
71: char ***pppsz_titles, uint16_t **ppi_ids);
172: free(p_ext->psz_name);
173: free(p_ext->psz_title);
174: free(p_ext->psz_author);
175: free(p_ext->psz_description);
176: free(p_ext->psz_shortdescription);
177: free(p_ext->psz_url);
178: free(p_ext->psz_version);
239: psz_package = strdup(p_ext->psz_name);
256: // Remove trailing 'c' --> try with .lua script
284: msg_Dbg(p_mgr, "Scanning Lua script %s", psz_filename);
291: msg_Dbg(p_this, "reading Lua script in a zip archive");
314: p_ext->psz_name = psz_script;
316: if(!p_ext->p_sys || !p_ext->psz_name)
318: free(p_ext->psz_name);
328: free(p_ext->psz_name);
```

```

416: p_ext->psz_title = strdup(luaL_checkstring(L, -1));
423: p_ext->psz_title = strdup(psz_script);
429: p_ext->psz_author = luaL_strdupornull(L, -1);
432: /* Get description */
433: lua_getfield(L, -1, "description");
434: p_ext->psz_description = luaL_strdupornull(L, -1);
437: /* Get short description */
438: lua_getfield(L, -1, "shortdesc");
439: p_ext->psz_shortdescription = luaL_strdupornull(L, -1);
444: p_ext->psz_url = luaL_strdupornull(L, -1);
449: p_ext->psz_version = luaL_strdupornull(L, -1);
487: free(p_ext->psz_name);
488: free(p_ext->psz_title);
489: free(p_ext->psz_url);
490: free(p_ext->psz_author);
491: free(p_ext->psz_description);
492: free(p_ext->psz_shortdescription);
493: free(p_ext->psz_version);
629: assert(p_ext->psz_name != NULL);
709: * @param pppsz_titles Pointer to NULL. All strings must be freed by the caller
716: char ***pppsz_titles, uint16_t **ppi_ids)
718: assert(*pppsz_titles == NULL);
746: "function menu() not found", p_ext->psz_name);
753: "function menu(): %s", p_ext->psz_name,
764: *pppsz_titles = (char**) calloc(i_size+1, sizeof(**pppsz_titles));
776: "the menu table is invalid!", p_ext->psz_name);
779: (*pppsz_titles)[i_idx] = strdup(luaL_checkstring(L, -1));
788: "did not return a table", p_ext->psz_name);
794: msg_Warn(p_mgr, "Script %s went completely foobar", p_ext->psz_name);
869: if(!strncmp(p_ext->psz_name, "zip://", 6))
876: if(vlclua_add_modules_path(L, p_ext->psz_name))
879: "search path for %s", p_ext->psz_name);
886: if(vlclua_dofile(VLC_OBJECT(p_mgr), L, p_ext->psz_name))
888: msg_Warn(p_mgr, "Error loading script %s: %s", p_ext->psz_name,
912: * Execute a function in a Lua script
938: "function %s() not found", p_ext->psz_name, psz_function);
957: "from script %s", type, psz_function, p_ext->psz_name);
969: "function %s(): %s", p_ext->psz_name, psz_function,
1004: "function trigger_menu() not found", p_ext->psz_name);
1014: "function trigger_menu(): %s", p_ext->psz_name,
1203: p_ext->psz_title);

```

## 27. assistant (2026-06-08T07:58:53.706Z)

Confirmed: the "Lua script" strings in extension.c are only debug messages ( `msg\_Dbg "Scanning Lua script %s"` ), not UI strings. So "Lua script" as a UI body must come from the GUI. The description/shortdesc are stored on `p\_ext`. Now let me confirm the macOS GUI angle, since the dialog format the user describes ("Type: Extensions" + ".lua" title + "Lua script" body) does not match the Qt ExtensionInfoDialog. Let me check the macOS interface for the extensions/plugins about dialog.

## 28. user (2026-06-08T07:58:56.597Z)

```

=== search macosx for extension about / Lua script / Type ===
modules/gui/macosx/windows/extensions/VLCUIWidgets.h
modules/gui/macosx/windows/extensions/VLCExtensionsManager.m
modules/gui/macosx/windows/extensions/VLCExtensionsDialogProvider.h
modules/gui/macosx/windows/extensions/VLCExtensionsDialogProvider.m
modules/gui/macosx/windows/video/VLCWindow.h
modules/gui/macosx/main/VLCMain.m
modules/gui/macosx/windows/addons/VLCAddonsWindowController.m
modules/gui/macosx/library/media-source/VLCMediaSourceDataSource.m
modules/gui/macosx/extensions/NSString+Helpers.h
modules/gui/macosx/playqueue/VLCPlayerController.h
modules/gui/macosx/extensions/NSString+Helpers.m

```

```

modules/gui/macosx/extensions/NSTableView+VLCAdditions.m
modules/gui/macosx/Makefile.am
modules/gui/macosx/windows/extensions/VLCUIWidgets.m
modules/gui/macosx/views/VLCBottomBarView.m
modules/gui/macosx/views/VLCVolumeSliderCell.m
modules/gui/macosx/windows/extensions/VLCExtensionsManager.h
modules/gui/macosx/panels/bookmarks/VLCBookmark.m
modules/gui/macosx/windows/convertandsave/VLCConvertAndSaveWindowController.m
modules/gui/macosx/main/VLCApplication.m
modules/gui/macosx/views/VLCPlaybackEndViewController.m
modules/gui/macosx/library/VLCLibraryWindow.m
modules/gui/macosx/library/VLCLibraryCollectionViewSupplementaryElementView.m
modules/gui/macosx/panels/VLCTimeSelectionPanelController.m
modules/gui/macosx/playqueue/VLCPlayQueueDataSource.m
modules/gui/macosx/views/VLCStatusNotifierView.m
modules/gui/macosx/library/VLCLibrarySegment.m
modules/gui/macosx/library/VLCLibraryWindowTitlesSidebarViewController.m
modules/gui/macosx/panels/VLCAudioEffectsWindowController.m
modules/gui/macosx/UI/MainMenu.xib
=== grep vlc.c lua module descriptions ===
85: set_shortcode(N_("Lua"))
86: set_description(N_("Lua interpreter"))
92: set_capability("interface", 0)
101: set_capability("interface", 0)
104: set_description(N_("Lua HTTP"))
110: set_capability("interface", 25)
111: set_description(N_("Command-line interface"))
129: set_capability("interface", 0)
131: set_description(N_("Lua Telnet"))
135: set_shortcode(N_("Lua Meta Fetcher"))
136: set_description(N_("Fetch meta data using lua scripts"))
137: set_capability("meta fetcher", 10)
141: set_shortcode(N_("Lua Meta Reader"))
142: set_description(N_("Read meta data using lua scripts"))
143: set_capability("meta reader", 10)
148: set_shortcode(N_("Lua Playlist"))
149: set_description(N_("Lua Playlist Parser Interface"))
150: set_capability("stream_filter", 2)
154: set_shortcode(N_("Lua Art"))
155: set_description(N_("Fetch artwork using lua scripts"))
156: set_capability("art finder", 10)
160: set_shortcode(N_("Lua Extension"))
161: set_description(N_("Lua Extension"))
162: add_shortcut("luaextension")
163: set_capability("extension", 1)
164: set_callbacks(Open_Extension, Close_Extension)
167: set_description(N_("Lua SD Module"))
169: set_capability("services_discovery", 0)
222: /* Lua scripts in user-specific data directory */
230: /* Tokenized Lua scripts in architecture-specific data directory */
233: /* Source Lua Scripts in architecture-independent data directory */
270: msg_Dbg(p_this, "Trying Lua scripts in %s", *ppsz_dir);

```

## 29. assistant (2026-06-08T07:59:05.011Z)

The module description for the extension submodule is "Lua Extension" (not "Lua script"). So the "Lua script" body text is a GUI-specific string. The user's described dialog format ? title "About <name>.lua", "Type: Extensions", body "Lua script" ? is a distinctive pattern. This is almost certainly the macOS VLCExtensionsDialogProvider or the macOS plugins window. But actually, reconsidering: "Type: Extensions" (plural, as a category name) with body "Lua script" ? this matches the \*\*macOS "Plugins and extensions" window's per-row About/Information\*\*. Let me check the macOS VLCAddonsWindowController and VLCExtensionsManager for this exact dialog, and the VLCUIWidgets (which is the macOS dialog API). Let me get the macOS dialog provider and the addons window.

### 30. user (2026-06-08T07:59:07.836Z)

```
=== VLCExtensionsManager.m: grep description/shortdesc/Lua/about ===
91: NSString *titleString = toNSStr(p_ext->psz_title);
101: char **ppsz_titles = NULL;
105: if (extension_GetMenu(p_extensions_manager, p_ext, &ppsz_titles, &pi_ids) ==
VLC_SUCCESS) {
106: for (int i = 0; ppsz_titles[i] != NULL; ++i) {
108: titleString = toNSStr(ppsz_titles[i]);
115: free(ppsz_titles[i]);
123: free(ppsz_titles);
127: p_ext->psz_title);
230: p_ext->psz_title, i_ext);
244: p_ext->psz_title, i_action);

=== search whole repo for 'Type: Extensions' or 'Extensions' type label in macosx prefs/about
===
modules/gui/macosx/preferences/prefs.m
modules/gui/macosx/preferences/prefs_widgets.m
modules/gui/macosx/preferences/VLCSimplePrefsController.m
```

### 31. assistant (2026-06-08T07:59:19.561Z)

The macOS extensions manager uses `psz\_title` for the menu but doesn't have an "About...Lua script" dialog with "Type: Extensions".

Let me reconsider the dialog the user describes more carefully. "Type: Extensions" with body "Lua script" and title "About <name>.lua" ? this is the **Qt PluginTab** module list after all, OR it's a generic VLC module-info dialog. Wait ? the Qt PluginTab has no About dialog. But there's another possibility: the user is on an older VLC 3.0.x Qt build where the **simple preferences "Plugins" tab** does have a description panel. Actually, the most direct match: **module\_get\_help()** returns help text, and for an extension the help is the module help ? which would be the help of the "Lua Extension" submodule = none, so it falls back to capability.

Let me step back. The distinctive "Type: Extensions" string. Let me search Qt prefs for the simple-prefs plugins about and the `extension` capability help mapping. Let me check `modules/gui/qt/dialogs/plugins.cpp` PluginTab more carefully ? whether double-click opens a generic dialog showing help, and where "Lua script" could come from. Let me get the raw plugins.cpp content directly.

### 32. user (2026-06-08T07:59:22.997Z)

```
=== plugins.cpp (3.0.x): grep for Type, help, name, description, doubleclick, About, Lua ===
35:#include "util/imagehelper.hpp"
163: qf_item << qfu(module_get_name(p_module, true))
164: << qfu(module_get_capability(p_module))
165: << QString::number(module_get_score(p_module));
310:static QIcon iconFromCategory(int type)
312: switch(type)
403: " Check descriptions for more details"),
448: addonsDelegate->setAnimator(new DelegateAnimationHelper(addonsView));
472: CONNECT(signalMapper, mapped(int), addonsModel, setTypeFilter(int));
517: switch(event->type())
630: name = qfu(p_ext->psz_name);
631: description = qfu(p_ext->psz_description);
632: shortdesc = qfu(p_ext->psz_shortdescription);
633: if(description.isEmpty())
634: description = shortdesc;
635: if(shortdesc.isEmpty() && !description.isEmpty())
636: shortdesc = description;
637: title = qfu(p_ext->psz_title);
638: author = qfu(p_ext->psz_author);
639: version = qfu(p_ext->psz_version);
640: url = qfu(p_ext->psz_url);
658: case SummaryRole:
```

```

659: return shortdesc;
797: returnval = qfu(p_entry->psz_name);
801: if (p_entry->psz_image_data)
804: pixmap.loadFromData(QByteArray::fromBase64(QByteArray(p_entry->psz_image_data
)),
823: case SummaryRole:
824: returnval = qfu(p_entry->psz_summary);
826: case DescriptionRole:
827: returnval = qfu(p_entry->psz_description);
829: case TypeRole:
830: returnval = QVariant((int) p_entry->e_type);
848: returnval = QVariant(qfu(p_entry->psz_version));
851: returnval = qfu(p_entry->psz_author);
854: returnval = qfu(p_entry->psz_source_uri);
860: list << qfu(p_file->psz_filename);
946: QColor AddonsListModel::getColorByAddonType(int i_type)
949: switch(i_type)
1001: i_type_filter = -1;
1005: void AddonsSortFilterProxyModel::setTypeFilter(int type)
1007: i_type_filter = type;
1025: if (i_type_filter > -1 &&
1026: item.data(AddonsListModel::TypeRole).toInt() != i_type_filter)
1092: index.data(ExtensionListModel::SummaryRole).toString());
1135: int i_type = index.data(AddonsListModel::TypeRole).toInt();
1137: /* Draw Background gradient by addon type */
1138: QColor backgroundColor = AddonsListModel::getColorByAddonType(i_type);
1213: scoreicon = ImageHelper::loadSvgToPixmap(":/addons/score.svg",
1335: void AddonItemDelegate::setAnimator(DelegateAnimationHelper *animator_)
1374: setWindowTitle(qtr("About") + " " + index.data(Qt::DisplayRole).toString());
1409: // Description
1411: label->setText(index.data(ExtensionListModel::SummaryRole).toString());
1457: setWindowTitle(qtr("About") + " " + index.data(Qt::DisplayRole).toString());
1490: // Type
1491: QString type = AddonsManager::getAddonType(
index.data(AddonsListModel::TypeRole).toInt());
1493: .arg(qtr("Type")).arg(type));
1513: .arg(index.data(AddonsListModel::SummaryRole).toString()));
1515: // Description
1516: QString description = index.data(AddonsListModel::DescriptionRole).toString();
1517: if (!description.isEmpty())

```

### 33. assistant (2026-06-08T07:59:34.730Z)

Now I have the definitive Qt 3.0.x source. Two key facts:

1. **ExtensionInfoDialog** (line 1374): `setWindowTitle( qtr("About") + " " + DisplayRole )``. The `DisplayRole` for an extension is its **name** (`psz_name``, the script filename, e.g. `myext.lua``). Line 1411 shows the **Description label = SummaryRole = shortdesc** (line 658-659). And critically, line 633-636 shows the fallback: if `shortdesc`` is empty, it uses `description``, and if `description`` is empty too, both stay empty.

2. The **"Type:"** field with body text only appears in **AddonInfoDialog** (line 1490-1493), NOT **ExtensionInfoDialog**.

So the user's described dialog ("Type: Extensions" + "Lua script") does NOT match the Qt 3.0.x **ExtensionInfoDialog**. This is a strong signal the user is describing a **different/older** dialog. Let me verify the `DisplayRole` (what fills the title after "About") and confirm whether older VLC (3.0.0-era) or a specific build shows "Lua script". Let me check the `DisplayRole` return and the `Qt::DisplayRole`` for **ExtensionListModel**, plus check if there's a generic module dialog. Let me also reconsider ? "Type: Extensions" strongly matches the **Qt Messages/Plugins** or possibly this is the VLC **mediainfo** "About".

Let me get the exact `DisplayRole` and the full **ExtensionInfoDialog** body to be precise, and check the macOS about-extension path which I haven't fully ruled out.

### 34. user (2026-06-08T07:59:37.629Z)

```
=== ExtensionListModel data(): DisplayRole + ExtensionInfoDialog full body (lines 640-720,
1360-1440) ===
 url = qfu(p_ext->psz_url);
 icon = loadPixmapFromData(p_ext->p_icondata, p_ext->i_icondata_size);
}

ExtensionListModel::ExtensionCopy::~ExtensionCopy()
{
 delete icon;
}

QVariant ExtensionListModel::ExtensionCopy::data(int role) const
{
 switch(role)
 {
 case Qt::DisplayRole:
 return title;
 case Qt::DecorationRole:
 if (!icon) return QPixmap(":/logo/vlc48.png");
 return *icon;
 case SummaryRole:
 return shortdesc;
 case VersionRole:
 return version;
 case AuthorRole:
 return author;
 case LinkRole:
 return url;
 case FilenameRole:
 return name;
 default:
 return QVariant();
 }
}

/* Extensions list model for the QListView */
ExtensionListModel::ExtensionListModel(QObject *parent)
 : QAbstractListModel(parent), EM(NULL)
{
}

ExtensionListModel::ExtensionListModel(QObject *parent, ExtensionsManager* EM_)
 : QAbstractListModel(parent), EM(EM_)
{
 // Connect to ExtensionsManager::extensionsUpdated()
 CONNECT(EM, extensionsUpdated(), this, updateList());

 // Load extensions now if not already loaded
 EM->loadExtensions();
}

ExtensionListModel::~ExtensionListModel()
{
 // Clear extensions list
 while(!extensions.isEmpty())
 delete extensions.takeLast();
}

void ExtensionListModel::updateList()
{
 ExtensionCopy *ext;
```

```

// Clear extensions list
while(!extensions.isEmpty())
{
 ext = extensions.takeLast();
 delete ext;
}

// Find new extensions
extensions_manager_t *p_mgr = EM->getManager();
if(!p_mgr)
 return;

vlc_mutex_lock(&p_mgr->lock);
extension_t *p_ext;
FOREACH_ARRAY(p_ext, p_mgr->extensions)
{
 ext = new ExtensionCopy(p_ext);
 extensions.append(ext);
}
FOREACH_END()
emit closeEditor(editor);
}

/* "More information" dialog */

ExtensionInfoDialog::ExtensionInfoDialog(const QModelIndex &index,
 intf_thread_t *p_intf,
 QWidget *parent)
 : QVLCDialog(parent, p_intf)
{
 // Let's be a modal dialog
 setWindowModality(Qt::WindowModal);

 // Window title
 setWindowTitle(qtr("About") + " " + index.data(Qt::DisplayRole).toString());

 // Layout
 QGridLayout *layout = new QGridLayout(this);

 // Icon
 QLabel *icon = new QLabel(this);
 QPixmap pix = index.data(Qt::DecorationRole).value<QPixmap>();
 Q_ASSERT(!pix.isNull());
 icon->setPixmap(pix);
 icon->setAlignment(Qt::AlignCenter);
 icon->setFixedSize(48, 48);
 layout->addWidget(icon, 1, 0, 2, 1);

 // Title
 QLabel *label = new QLabel(index.data(Qt::DisplayRole).toString(), this);
 QFont font = label->font();
 font.setBold(true);
 font.setPointSizeF(font.pointSizeF() * 1.3f);
 label->setFont(font);
 layout->addWidget(label, 0, 0, 1, -1);

 // Version
 label = new QLabel("" + qtr("Version") + "", this);
 layout->addWidget(label, 1, 1, 1, 1, Qt::AlignBottom);
 label = new QLabel(index.data(ExtensionListModel::VersionRole).toString(), this);
 layout->addWidget(label, 1, 2, 1, 2, Qt::AlignBottom);

 // Author
 label = new QLabel("" + qtr("Author") + "", this);
 layout->addWidget(label, 2, 1, 1, 1, Qt::AlignTop);

```

```

label = new QLabel(index.data(ExtensionListModel::AuthorRole).toString(), this);
layout->addWidget(label, 2, 2, 1, 2, Qt::AlignTop);

// Description
label = new QLabel(this);
label->setText(index.data(ExtensionListModel::SummaryRole).toString());
label->setWordWrap(true);
label->setOpenExternalLinks(true);
layout->addWidget(label, 4, 0, 1, -1);

// URL
label = new QLabel("" + qtr("Website") + "", this);
layout->addWidget(label, 5, 0, 1, 2);
label = new QLabel(QString("%2")
 .arg(index.data(ExtensionListModel::LinkRole).toString())
 .arg(index.data(ExtensionListModel::LinkRole).toString())
 , this);
label->setOpenExternalLinks(true);
layout->addWidget(label, 5, 2, 1, -1);

// Script file
label = new QLabel("" + qtr("File") + "", this);
layout->addWidget(label, 6, 0, 1, 2);
QLineEdit *line =
 new QLineEdit(index.data(ExtensionListModel::FilenameRole).toString(), this);
line->setReadOnly(true);
layout->addWidget(line, 6, 2, 1, -1);

// Close button
QDialogButtonBox *group = new QDialogButtonBox(this);
QPushButton *closeButton = new QPushButton(qtr("&Close"));
group->addButton(closeButton, QDialogButtonBox::RejectRole);
BUTTONACT(closeButton, close());

layout->addWidget(group, 7, 0, 1, -1);

```

### 35. assistant (2026-06-08T07:59:49.468Z)

This is the definitive answer for the Qt "More information" / About dialog. Key findings now crystal clear:

- **Qt 3.0.x** `ExtensionInfoDialog` (the "About <extension>" / "More information" dialog reached from Tools ? Plugins and extensions ? Extensions tab):
  - Title = `"About " + DisplayRole` where `DisplayRole = `title`` (`psz_title``, from descriptor `title``, defaulting to script filename).
  - Body shows `Version`, `Author`, a `Description` label (= `SummaryRole = `shortdesc``), `Website (url)`, `File (psz_name)`.
  - It does **NOT** show a "Type:" field and does **NOT** show "Lua script". The descriptor's `description`/`shortdesc`` DOES populate this dialog (via the `shortdesc`, with description as fallback per lines 633-636).

So the dialog the user describes ("Type: Extensions", body "Lua script") is **NOT** the Qt `ExtensionInfoDialog`. The "Type:" field + a generic capability/help body is characteristic of a **module-info** style dialog. Given "Lua script" matches no VLC 3.x `set_help/set_description` string I found (the extension submodule description is "Lua Extension"), and "Type: Extensions" capitalizes "Extensions" as a category ? this strongly matches the **macOS VLC** interface's per-item info, OR a third-party/older build. Let me do a focused search for the exact UI string "Lua script" appearing as a help/about body and "Type" in the macOS or addons context, and check the addons `VLCAddonsWindowController`.

### 36. user (2026-06-08T07:59:56.490Z)

=== Does ANY non-debug source set a UI string 'Lua script'? Re-list with context ===  
 --- modules/lua/extension.c ---

```

256: // Remove trailing 'c' --> try with .lua script
284: msg_Dbg(p_mgr, "Scanning Lua script %s", psz_filename);
291: msg_Dbg(p_this, "reading Lua script in a zip archive");
912: * Execute a function in a Lua script
--- modules/lua/meta.c ---
2: * meta.c: Get meta/artwork using lua scripts
--- modules/lua/services_discovery.c ---
2: * services_discovery.c : Services discovery using lua scripts
172: // This module can be used to load lua script not registered
--- modules/lua/stream_filter.c ---

=== addons fsstorage.c 'Lua script' context (this registers addon types) ===
47: * Local prototypes
56: addon_type_t t;
59: { ADDON_EXTENSION, ADDONS_SCRIPTS_DIR DIR_SEP "extensions" },
60: { ADDON_PLAYLIST_PARSER, ADDONS_SCRIPTS_DIR DIR_SEP "playlist" },
74: const addon_type_t e_type,
99: static char * getAddonInstallDir(addon_type_t t)
153: int i_current_node_type;
171: while(!b_done && (i_current_node_type = xml_ReaderNextNode(p_xml_reader, &p_node)) >
0)
173: switch(i_current_node_type)
250: p_entry->e_type = ADDON_SKIN2;
255: p_entry->psz_description = strdup("Skins2 theme");
262: p_entry->psz_description = strdup("Skins2 theme");
276: const addon_type_t e_type,
283: if (p_file->e_filetype == e_type
291: static int ListScripts(addons_finder_t *p_finder, addon_type_t type)
293: char *psz_dir = getAddonInstallDir(type);
304: if (FileBelongsToManagedAddon(p_finder, type, psz_file))
308: p_entry->e_type = type;
311: p_entry->psz_description = strdup("Lua script");
325: addon_type_t types[] = {
326: ADDON_EXTENSION,
327: ADDON_PLAYLIST_PARSER,
332: unsigned int i_type = 0;
337: while(i_type < ARRAY_SIZE(types))
339: ListScripts(p_finder, types[i_type++]);
445: switch(p_file->e_filetype)
447: case ADDON_EXTENSION:
448: case ADDON_PLAYLIST_PARSER:
463: char *psz_dir = getAddonInstallDir(p_file->e_filetype);
608: fprintf(p_catalog, "\t\t<addon source=\"%s\" type=\"%s\" id=\"%s\" "

```

### 37. assistant (2026-06-08T08:00:05.550Z)

Found it definitively. Line 311 in `modules/misc/addons/fsstorage.c`:

```

```c
p_entry->psz_description = strdup("Lua script");
```

```

This is the **addons "file-system storage" finder** (`ListScripts`). It is the `addons_finder` capability that scans local `extensions/` and `playlist/` directories and presents installed Lua scripts as **addon entries** with a **hardcoded** `psz_description = "Lua script"`. The entry's `e_type` is `ADDON_EXTENSION` (line 308/326).

So the dialog the user describes ? title "About <name>.lua", **\*\*\*Type: Extensions\*\*\***, body **\*\*\*Lua script\*\*\*** ? is the **AddonInfoDialog** (Add-ons tab, lines 1457-1517 of `plugins.cpp`), NOT the `ExtensionInfoDialog`. Let me confirm the `AddonInfoDialog` title (`DisplayRole = addon name = filename .lua`), the Type mapping (`getAddonType(ADDON_EXTENSION)` ? "Extensions"), and the description = the hardcoded "Lua script". Let me verify `getAddonType` returns "Extensions" for `ADDON_EXTENSION` and dump the `AddonInfoDialog` body and addon `NameRole`.

### 38. user (2026-06-08T08:00:11.183Z)

```

=== AddonsManager::getAddonType mapping (which returns 'Extensions') ===
modules/gui/macros/windows/addons/VLCAddonsWindowController.m
--- grep AddonsManager.cpp for getAddonType + ADDON_EXTENSION + Extensions ---
81:QString AddonsManager::getAddonType(int i_type)
86: return qtr("Skins");
87: case ADDON_PLAYLIST_PARSER:
88: return qtr("Playlist parsers");
89: return qtr("Service Discovery");
90: return qtr("Interfaces");
91: return qtr("Art and meta fetchers");
92: case ADDON_EXTENSION:
93: return qtr("Extensions");
94: return qtr("Unknown");

=== AddonInfoDialog body + AddonsListModel NameRole/DisplayRole (plugins.cpp 1455-1540, 790-835)
===

// Window title
setWindowTitle(qtr("About") + " " + index.data(Qt::DisplayRole).toString());

// Layout
QGridLayout *layout = new QGridLayout(this);
QLabel *label;

// Icon
QLabel *iconLabel = new QLabel(this);
iconLabel->setFixedSize(100, 100);
QPixmap icon = index.data(Qt::DecorationRole).value<QPixmap>();
icon.scaled(iconLabel->size(), Qt::KeepAspectRatio, Qt::SmoothTransformation);
iconLabel->setPixmap(icon);
iconLabel->setAlignment(Qt::AlignCenter | Qt::AlignTop);
layout->addWidget(iconLabel, 1, 0, 2, 1);

// Title
label = new QLabel(index.data(Qt::DisplayRole).toString(), this);
QFont font = label->font();
font.setBold(true);
font.setPointSizeF(font.pointSizeF() * 1.3f);
label->setFont(font);
layout->addWidget(label, 0, 0, 1, -1);

// HTML Content on right side
QTextEdit *textContent = new QTextEdit();
textContent->viewport()->setAutoFillBackground(false);
textContent->setAcceptRichText(true);
textContent->setBackgroundRole(QPalette::Window);
textContent->setFrameStyle(QFrame::NoFrame);
textContent->setAutoFillBackground(false);
textContent->setReadOnly(true);
layout->addWidget(textContent, 1, 1, 4, -1);

// Type
QString type = AddonsManager::getAddonType(index.data(AddonsListModel::TypeRole).toInt());
textContent->append(QString("%1: %2
")
 .arg(qtr("Type")).arg(type));

// Version
QString version = index.data(ExtensionListModel::VersionRole).toString();
if (!version.isEmpty())
{
 textContent->append(QString("%1: %2
")
 .arg(qtr("Version")).arg(version));
}

// Author

```

```

QString author = index.data(ExtensionListModel::AuthorRole).toString();
if (!author.isEmpty())
{
 textContent->append(QString("%1: %2
")
 .arg(qtr("Author")).arg(author));
}

// Summary
textContent->append(QString("%1
\n")
 .arg(index.data(AddonsListModel::SummaryRole).toString()));

// Description
QString description = index.data(AddonsListModel::DescriptionRole).toString();
if (!description.isEmpty())
{
 textContent->append(QString("<hr/>\n%1")
 .arg(description.replace("\n", "
")));
}

// URL
QString sourceUrl = index.data(ExtensionListModel::LinkRole).toString();
if (!sourceUrl.isEmpty())
{
 label = new QLabel("" + qtr("Website") + ":", this);
 layout->addWidget(label, 5, 0, 1, 2);
 label = new QLabel(QString("%2")
 .arg(sourceUrl).arg(sourceUrl), this);
 label->setOpenExternalLinks(true);
 layout->addWidget(label, 5, 2, 1, -1);
}

// Script files
QList<QVariant> list = index.data(ExtensionListModel::FilenameRole).toList();
if (!list.empty())
{
 label = new QLabel("" + qtr("Files") + ":", this);
 layout->addWidget(label, 6, 0, 1, 2);
}

```

### 39. assistant (2026-06-08T08:00:20.991Z)

Fully confirmed. Now I have the complete, precise chain. The dialog the user describes is the **Qt `AddonInfoDialog`** (the "About" dialog in the **Add-ons** sub-tab of Tools ? Plugins and extensions, reached by selecting an installed local Lua script and clicking "More" / its info action):

- Title: `"About " + DisplayRole``. For a local-disk addon, `DisplayRole` = the addon **name** = the script filename, hence **"About <name>.lua"**.
- `Type:` = `getAddonType(ADDON_EXTENSION)` = "Extensions" (addons_manager.cpp:95-96).`
- The body **"Lua script"** = the addon entry's `psz_description``, which the `fsstorage.c`` addons finder **hardcodes** to `"Lua script`` for every locally-discovered Lua script (`fsstorage.c:311`). The `DescriptionRole` appended at the bottom (line ~1516) is also this hardcoded `"Lua script"`.

This is a generic per-addon-type string an individual extension script **cannot** control ? it is set by the file-system addons finder, not by the script's `descriptor()``. The descriptor's `description`/shortdesc`` are NOT read by this addons-finder path (it just scans files on disk; it does not parse the Lua descriptor).

Let me do one final verification: confirm the addon `psz_name`/DisplayRole` for `fsstorage` entries is the filename (so the title is `"<name>.lua"`), and grab a couple of canonical wiki/forum URLs for citations.

### 40. user (2026-06-08T08:00:25.628Z)

=== `fsstorage.c ListScripts`: how `psz_name` is set for each script entry (lines 291-320) ===

```

static int ListScripts(addons_finder_t *p_finder, addon_type_t type)
{
 char *psz_dir = getAddonInstallDir(type);
 if (! psz_dir) return VLC_EGENERIC;

 char **ppsz_list = NULL;
 int i_count = vlc_scandir(psz_dir, &ppsz_list, ListScript_filter, NULL);

 for (int i=0; i< i_count; i++)
 {
 char *psz_file = ppsz_list[i];
 if(!psz_file)
 break;
 if (FileBelongsToManagedAddon(p_finder, type, psz_file))
 continue;
 addon_entry_t *p_entry = addon_entry_New();
 p_entry->e_state = ADDON_INSTALLED;
 p_entry->e_type = type;
 p_entry->e_flags |= ADDON_BROKEN;
 p_entry->psz_name = strdup(psz_file);
 p_entry->psz_description = strdup("Lua script");

 ARRAY_APPEND(p_finder->entries, p_entry);
 free(psz_file);
 }

 free(ppsz_list);
 free(psz_dir);

 return VLC_SUCCESS;
}

```

=== AddonsListModel DisplayRole/NameRole (plugins.cpp ~790-800) ===

QVariant AddonsListModel::Addon::data( int role ) const

```

{
 QVariant returnval;

 vlc_mutex_lock(&p_entry->lock);
 switch(role)
 {
 case Qt::DisplayRole:
 {
 returnval = qfu(p_entry->psz_name);
 break;
 }
 case Qt::DecorationRole:

```

#### 41. assistant (2026-06-08T08:00:29.574Z)

Fully confirmed end-to-end. `p\_entry->psz\_name = strdup(psz\_file)` (the filename, e.g. `myext.lua`), and DisplayRole returns `psz\_name`. So the AddonInfoDialog title = "About myext.lua", Type = "Extensions", body = "Lua script". The chain is airtight.

Let me grab a couple of canonical wiki citation URLs to round out the sources, then submit.

#### 42. user (2026-06-08T08:00:35.624Z)

Web search results for query: "VideoLAN wiki Documentation Lua Extensions descriptor dialog add\_html add\_button add\_label widget API"

Links: [{"title":"Dialog Module - VLC Lua Docs","url":"https://vlc.verg.ca/m/dialog/"},"{"title":"Lua extensions: selected item API for dialog dropdown widget (#6548) · Issues · VideoLAN / VLC · GitLab","url":"https://trac.videolan.org/vlc/ticket/6548"},"{"title":"Altering \"time\", adding

hotkeys via LUA - The VideoLAN

```
Forums", "url": "https://forum.videolan.org/viewtopic.php?t=100885"}, {"title": "Extension Scripts -
VLC Lua Docs", "url": "https://vlc.verg.ca/t/extensions/"}, {"title": "Documentation: Modules/lua -
VideoLAN Wiki", "url": "https://wiki.videolan.org/Documentation:Modules/lua/"}, {"title": "Any
documentation on extensions? - The VideoLAN
Forums", "url": "https://forum.videolan.org/viewtopic.php?t=108385"}, {"title": "Extensions ? VLC
Desktop User Documentation 3.0 documentation", "url": "https://docs.videolan.me/vlc-
user/desktop/3.0/en/addons/extensions.html"}, {"title": "VLC Lua
Docs", "url": "https://vlc.verg.ca/"}, {"title": "vlc-lua-docs/t/extensions/index.md at master ·
verghost/vlc-lua-docs", "url": "https://github.com/verghost/vlc-lua-
docs/blob/master/t/extensions/index.md"}, {"title": "Where is the API document for lua (#26924) ·
Issues · VideoLAN / VLC ·
GitLab", "url": "https://code.videolan.org/videolan/vlc/-/issues/26924"}]
```

I found documentation on the VideoLAN Lua API for dialog widgets. Here's what the search revealed:

## Dialog Widget API - Key Methods

The `add_button` method adds a button widget to the dialog with text and a callback function.

The `add_label` method adds a text label widget to the dialog.

The `add_html` method adds a rich text label widget formatted with basic HTML (such as `<i>` and `<h1>`) to the dialog.

## Extension Descriptor Function

The descriptor function is one of the most important special functions in VLC Lua scripts, and in extensions it is required for the extension to be registered by VLC. This function only needs to return a table of information, which can have fields like title (the only mandatory field).

## Dialog Access

Extension scripts are the only kind that have access to the dialog Lua module, which allows developers to create a new dialog window.

The primary source for this documentation appears to be the VLC Lua Docs at `vlc.verg.ca`, which provides detailed API reference information for VLC extension development.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

### 43. assistant (2026-06-08T08:01:06.238Z)

I have all the facts verified directly from VLC 3.0.x source. The `add_html` rendering basic HTML (`<i>`, `<h1>`) is confirmed by the VLC Lua docs. I have everything needed to answer precisely. Submitting the structured output.

### 44. user (2026-06-08T08:01:13.606Z)

Structured output provided successfully

### 45. assistant (2026-06-08T08:01:27.155Z)

I've completed the research and submitted the structured answer. Here is the summary of what I found, verified directly against the VLC 3.0.x source via the GitLab/GitHub mirror.

## The key finding: two different dialogs

The dialog the user is describing ? title "About <name>.lua", "Type: Extensions", body "Lua script" ? is **NOT** the Extensions-tab info dialog. It is the **Add-ons** sub-tab's

`AddonInfoDialog`, and every piece of its text is fabricated by VLC's filesystem addons  
\*finder\*, not by the script's `descriptor()`:

```
- **Title "About <name>.lua" ? `setWindowTitle(qtr("About") + " " + DisplayRole)`; DisplayRole
= `p_entry->psz_name`, which `modules/misc/addons/fsstorage.c` ListScripts() sets to the bare
script filename.
- **"Type: Extensions" ? `getAddonType(ADDON_EXTENSION)` returns `qtr("Extensions")` in
`modules/gui/qt/managers/addons_manager.cpp:95-96`.
- **Body "Lua script" ? hardcoded at `modules/misc/addons/fsstorage.c:311`:
`p_entry->psz_description = strdup("Lua script");`. The finder only `scandir()`s the directory;
it never parses the Lua descriptor. So this string is uncontrollable by the script. (The "Lua
script" occurrences in `extension.c` are only `msg_Dbg` logs; the lua module's own extension-
capability description is "Lua Extension" in `vlc.c:160-161`.)
```

## Where descriptor description/shortdesc DO show

In the separate \*\*Extensions-tab\*\* `ExtensionInfoDialog` (plugins.cpp ~1360-1440): the  
"Description" label = `SummaryRole` = `shortdesc` (with description as fallback, plugins.cpp  
~633-636), plus Version/Author/Website/File. That dialog has no "Type:" line and never shows  
"Lua script". Title there uses `title` (psz\_title, defaulting to the filename).

## Most reliable author-controlled help

Render help inside the extension's own `vlc.dialog`: an `add\_button("Help", cb)` plus an  
`add\_html(...)` widget (basic HTML / multi-line, confirmed via VLC Lua docs; maps to  
`EXTENSION\_WIDGET\_HTML`). Full verified API from `modules/lua/libs/dialog.c`: dialog `add\_button`  
/`add\_label`/`add\_html`/`add\_text\_input`/`add\_password`/`add\_check\_box`/`add\_dropdown`/`add\_list`/`add\_image`/`ad`  
`d\_spin\_icon`, plus `show`/`hide`/`delete`/`set\_title`/`update`/`del\_widget`; widgets `set\_text`/`get\_text`/`se`  
`t\_checked`/`get\_checked`/`add\_value`/`get\_value`/`clear`/`get\_selection`/`animate`/`stop`.

One flagged uncertainty: the C source proves `add\_html` \*exists\* but does not document its HTML  
tag coverage; that "basic HTML" claim rests on the VLC Lua docs and the Qt/macOS rich-text  
renderer behavior.